

MiraNote Architecture Snapshot v0.1

Status: Living document. Updated 2026-05-28.

1. Runtime topology

Three surfaces talk to one backend. The iOS app captures voice and short text. The web app is the workspace where notes live, are organized, and queried. The Discord bot is a low-friction Q&A channel for the team. All three surfaces speak HTTPS to the `api/` service, which fronts the LLM provider and the storage layer.

2. POC layer

Before any feature lands in the `api/` service, it is prototyped under `poc/` as a standalone FastAPI app. Current POCs: voice-to-text (Whisper + LLM correction), text-clean-expand (clean and expand free-form text), and chatbot (multi-turn conversation with native function calling). The chatbot POC reads documents in `DOCS_ROOT` via three tools: `list_docs`, `read_doc`, `search_docs`.

3. LLM provider abstraction

Every POC uses the `openai` Python SDK against any OpenAI-compatible endpoint. The default is DeepSeek v4-flash (see ADR-002). Switching to Gemini, OpenAI, or Moonshot is a one-line `.env` change. No DeepSeek-specific code lives in the POCs.

4. Open questions for v0.2

Persistent sessions (SQLite vs Redis), streaming responses (SSE), retrieval over the shared knowledge base (vector index), and auth (header token vs OIDC). None of these block the current POCs but all show up on the Q3 roadmap.